Shortest Path Problems and Informed search

Christos Dimitrakakis

March 5, 2026

Outline

The Shortest Path Problem
- The shortest path problem
- Heuristic Search
- Upper and lower bounds algorithms

General weight shortest path
- General weight shortest path

The costs of a path

At time t:

- We are in state $s_t \in S$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The cost of a policy $\pi : S \to A$.

- A policy π specifies a path $s_1, \ldots$ with $s_{k+1} = \tau(s_k, \pi(s_k))$

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The cost of a policy $\pi : S \to A$.

- A policy π specifies a path $s_1, \ldots$ with $s_{k+1} = \tau(s_k, \pi(s_k))$
- Following π from state s_1 has a total cost $C^\pi(s_1) \triangleq \sum_{k=1}^{t} c(s_k, \pi(s_k))$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The cost of a policy $\pi : S \to A$.

- A policy π specifies a path $s_1, \ldots$ with $s_{k+1} = \tau(s_k, \pi(s_k))$
- Following π from state s_1 has a total cost $C^\pi(s_1) \triangleq \sum_{k=1}^{t} c(s_k, \pi(s_k))$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The cost of a policy $\pi : S \to A$.

- A policy π specifies a path $s_1, \ldots$ with $s_{k+1} = \tau(s_k, \pi(s_k))$
- Following π from state s_1 has a total cost $C^\pi(s_1) \triangleq \sum_{k=1}^{t} c(s_k, \pi(s_k))$.

The cost of a path (alternative)

- We can instead use costs for going from state x to state y: $c(\langle x, y \rangle)$.

The costs of a path

At time t:

- We are in state $s_t \in S$.
- We take some action $a_t \in A$.
- Then we pay cost $c_t = c(s_t, a_t)$ and...
- .. move to state $s_{t+1} = \tau(s_t, a_t)$.

The cost of a policy $\pi : S \to A$.

- A policy π specifies a path $s_1, \ldots$ with $s_{k+1} = \tau(s_k, \pi(s_k))$
- Following π from state s_1 has a total cost $C^\pi(s_1) \triangleq \sum_{k=1}^{t} c(s_k, \pi(s_k))$.

The cost of a path (alternative)

- We can instead use costs for going from state x to state y: $c(\langle x, y \rangle)$.
- Following a path h has a total cost $C(h) \triangleq \sum_{\langle x,y \rangle \in h} c(\langle x, y \rangle)$

The shortest path problem

Input:

- State space S.

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.
- start states $X \subset S$

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.
- start states $X \subset S$
- goal states $Y \subset S$

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.
- start states $X \subset S$
- goal states $Y \subset S$
- costs $c : S \times A \to \mathbb{R}$

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.
- start states $X \subset S$
- goal states $Y \subset S$
- costs $c : S \times A \to \mathbb{R}$

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.
- start states $X \subset S$
- goal states $Y \subset S$
- costs $c : S \times A \to \mathbb{R}$

Output

Either:

- Find a path h from X to Y so that $C(h) \leq C(h')$ for all h'
- Find a policy $\pi : S \to A$ that finds a shortest path for every starting state.

The shortest path problem

Input:

- State space S.
- transition function $\tau : S \times A \to S$.
- start states $X \subset S$
- goal states $Y \subset S$
- costs $c : S \times A \to \mathbb{R}$

Output

Either:

- Find a path h from X to Y so that $C(h) \leq C(h')$ for all h'
- Find a policy $\pi : S \to A$ that finds a shortest path for every starting state.

Notes

- If the path/policy does not reach a goal, the cost is infinite.
- We can maximise rewards instead of minimising costs.

Formalising the shortest path problem

The cost from state s of a policy that reaches a goal in Y is

$$C^{\pi}(s) \triangleq \sum_{i=1}^{\infty} c[s_i, \pi(s_i)], \qquad s_{i+1} = \tau[s_i, \pi(s_i)], \quad s_1 = s$$

where for every $y \in Y$, $c(s, a) = 0$ and $\tau(y, a) = y$ for all actions.

Formalising the shortest path problem

The cost from state s of a policy that reaches a goal in Y is

$$C^{\pi}(s) \triangleq \sum_{i=1}^{\infty} c[s_i, \pi(s_i)], \qquad s_{i+1} = \tau[s_i, \pi(s_i)], \quad s_1 = s$$

where for every $y \in Y$, $c(s, a) = 0$ and $\tau(y, a) = y$ for all actions.

- We can calculate this recursively:

$$C^{\pi}(s) = \sum_{t=1}^{\infty} c[s_t, \pi(s_t)]$$

Formalising the shortest path problem

The cost from state s of a policy that reaches a goal in Y is

$$C^\pi(s) \triangleq \sum_{i=1}^{\infty} c[s_i, \pi(s_i)], \qquad s_{i+1} = \tau[s_i, \pi(s_i)], \quad s_1 = s$$

where for every $y \in Y$, $c(s, a) = 0$ and $\tau(y, a) = y$ for all actions.

- We can calculate this recursively:

$$\begin{aligned} C^\pi(s) &= \sum_{t=1}^{\infty} c[s_t, \pi(s_t)] \\ &= c[s, \pi(s)] + \sum_{t=2}^{\infty} c[s_t, \pi(s_t)] \end{aligned}$$

Formalising the shortest path problem

The cost from state s of a policy that reaches a goal in Y is

$$C^\pi(s) \triangleq \sum_{i=1}^{\infty} c[s_i, \pi(s_i)], \qquad s_{i+1} = \tau[s_i, \pi(s_i)], \quad s_1 = s$$

where for every $y \in Y$, $c(s, a) = 0$ and $\tau(y, a) = y$ for all actions.

- We can calculate this recursively:

$$\begin{aligned} C^\pi(s) &= \sum_{t=1}^{\infty} c[s_t, \pi(s_t)] \\ &= c[s, \pi(s)] + \sum_{t=2}^{\infty} c[s_t, \pi(s_t)] \\ &= c[s, \pi(s)] + C^\pi\{\tau[s, \pi(s)]\}. \end{aligned}$$

Formalising the shortest path problem

The cost from state s of a policy that reaches a goal in Y is

$$C^\pi(s) \triangleq \sum_{i=1}^{\infty} c[s_i, \pi(s_i)], \qquad s_{i+1} = \tau[s_i, \pi(s_i)], \quad s_1 = s$$

where for every $y \in Y$, $c(s, a) = 0$ and $\tau(y, a) = y$ for all actions.

- We can calculate this recursively:

$$\begin{aligned} C^\pi(s) &= \sum_{t=1}^{\infty} c[s_t, \pi(s_t)] \\ &= c[s, \pi(s)] + \sum_{t=2}^{\infty} c[s_t, \pi(s_t)] \\ &= c[s, \pi(s)] + C^\pi\{\tau[s, \pi(s)]\}. \end{aligned}$$

- The same idea applies for the shortest path

$$C^*(s) \triangleq \min_\pi C^\pi(s) = \min_a \{c[s, a] + C^*[\tau(s, a)]\}. \tag{1}$$

Dijkstra's shortest path algorithm: backward search

Dijkstra(Y, x) # Y are goal states Y, x is starting state.
$C(s) = 0$ for $s \in Y$, $C(s) = \infty$ for $s \notin Y$, $F_0 = Y$
for $t = 0, 1, \ldots$ **do**
  $s_t = \arg\min_{s \in F_t} C(s)$
  for $s \in \mathrm{Pa}(s_t)$ **do**
    $\pi(s) = \arg\min_a c(s, a) + C(\tau(s, a))$
    $C(s) = \min_a c(s, a) + C(\tau(s, a))$
  end for
  $F_{t+1} = F_t \setminus \{s_t\} \cup \mathrm{Pa}(s_t)$
  if $F_{t+1} = \emptyset$ or $x \in F_t$ **then**
    return π, C
  end if
end for

Algorithm idea

- Start from goal states
- Go back one step each time, adding the cost.
- Stop whenever there are no more states to go back to, or if we reach the start state.

Optimality proof

Theorem

$C(s) = C^*(s)$

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.
- For any other s, we will show that if s' is a next state and $C(s') \leq C^*(s')$ then $C(s) \leq C^*(s)$.

$$C(s) = \min_a \{c(s,a) + C(\tau(s,a))\} \qquad \text{(by definition)}$$

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.
- For any other s, we will show that if s' is a next state and $C(s') \leq C^*(s')$ then $C(s) \leq C^*(s)$.

$$\begin{aligned} C(s) &= \min_a \{c(s,a) + C(\tau(s,a))\} && \text{(by definition)} \\ &\leq \min_a \{c(s,a) + C^*(\tau(s,a))\} && \text{(by induction)} \end{aligned}$$

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.
- For any other s, we will show that if s' is a next state and $C(s') \leq C^*(s')$ then $C(s) \leq C^*(s)$.

$$
\begin{aligned}
C(s) &= \min_a \{c(s,a) + C(\tau(s,a))\} && \text{(by definition)} \\
&\leq \min_a \{c(s,a) + C^*(\tau(s,a))\} && \text{(by induction)} \\
&\leq \min_a \left\{c(s,a) + C^{\pi'}(\tau(s,a))\right\}, \qquad \forall \pi' && \text{(by optimality)}
\end{aligned}
$$

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.
- For any other s, we will show that if s' is a next state and $C(s') \leq C^*(s')$ then $C(s) \leq C^*(s)$.

$$
\begin{aligned}
C(s) &= \min_a \{c(s,a) + C(\tau(s,a))\} && \text{(by definition)} \\
&\leq \min_a \{c(s,a) + C^*(\tau(s,a))\} && \text{(by induction)} \\
&\leq \min_a \left\{c(s,a) + C^{\pi'}(\tau(s,a))\right\}, \qquad \forall \pi' && \text{(by optimality)} \\
&\leq C^{\pi}(s), \qquad \forall \pi.
\end{aligned}
$$

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.
- For any other s, we will show that if s' is a next state and $C(s') \leq C^*(s')$ then $C(s) \leq C^*(s)$.

$$
\begin{aligned}
C(s) &= \min_a \{c(s,a) + C(\tau(s,a))\} && \text{(by definition)} \\
&\leq \min_a \{c(s,a) + C^*(\tau(s,a))\} && \text{(by induction)} \\
&\leq \min_a \left\{c(s,a) + C^{\pi'}(\tau(s,a))\right\}, \qquad \forall \pi' && \text{(by optimality)} \\
&\leq C^{\pi}(s), \qquad \forall \pi.
\end{aligned}
$$

For the optimal policy π^*, $C^{\pi^*}(s) = C^*(s) \leq C^{\pi}(s)$ for any π. Finally,

Optimality proof

Theorem

$C(s) = C^*(s)$

Proof

- If $s \in Y$, then $C(s) = 0 = C^*(s)$.
- For any other s, we will show that if s' is a next state and $C(s') \leq C^*(s')$ then $C(s) \leq C^*(s)$.

$$
\begin{aligned}
C(s) &= \min_a \{c(s,a) + C(\tau(s,a))\} && \text{(by definition)} \\
&\leq \min_a \{c(s,a) + C^*(\tau(s,a))\} && \text{(by induction)} \\
&\leq \min_a \left\{c(s,a) + C^{\pi'}(\tau(s,a))\right\}, \qquad \forall \pi' && \text{(by optimality)} \\
&\leq C^{\pi}(s), \qquad \forall \pi.
\end{aligned}
$$

For the optimal policy π^*, $C^{\pi^*}(s) = C^*(s) \leq C^{\pi}(s)$ for any π. Finally,

$$C^*(s) \leq C^{\pi}(s) = C(s) \leq C^*(s),$$

since $C^{\pi}(s) = C(s)$ for the policy returned by the algorithm.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.

Finding all optimal paths to Y.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.

Finding all optimal paths to Y.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.

Finding all optimal paths to Y.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.
- This is because of the greedy step in which states are searched.

Finding all optimal paths to Y.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.
- This is because of the greedy step in which states are searched.
- This version is used in search and practical papers.

Finding all optimal paths to Y.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.
- This is because of the greedy step in which states are searched.
- This version is used in search and practical papers.

Finding all optimal paths to Y.

- Stops when no more updates are possible.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.
- This is because of the greedy step in which states are searched.
- This version is used in search and practical papers.

Finding all optimal paths to Y.

- Stops when no more updates are possible.
- The costs along all paths are correct.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.
- This is because of the greedy step in which states are searched.
- This version is used in search and practical papers.

Finding all optimal paths to Y.

- Stops when no more updates are possible.
- The costs along all paths are correct.
- This is because of the stopping condition.

Properties of Dijkstra's algorithm

The properties of the algorithm depend on some details.

Finding a shortest path from X to Y.

- Stops when a starting state is found.
- The costs along that optimal path are correct.
- Costs of any other state may be incorrect.
- This is because of the greedy step in which states are searched.
- This version is used in search and practical papers.

Finding all optimal paths to Y.

- Stops when no more updates are possible.
- The costs along all paths are correct.
- This is because of the stopping condition.
- This version is used in theoretical algorithmic papers.

Partial graphs

- Why do we need search?

Partial graphs

- Why do we need search?
- We do not want to calculate on the whole graph

Partial graphs

- Why do we need search?
- We do not want to calculate on the whole graph
- We use search to find the shortest path more efficiently (perhaps).

Partial graphs

- Why do we need search?
- We do not want to calculate on the whole graph
- We use search to find the shortest path more efficiently (perhaps).
- We denote the total cost of some path $s_1, \ldots, s_t$ as:

$$C(s_1, \ldots, s_t)$$

Partial graphs

- Why do we need search?
- We do not want to calculate on the whole graph
- We use search to find the shortest path more efficiently (perhaps).
- We denote the total cost of some path $s_1, \ldots, s_t$ as:

$$C(s_1, \ldots, s_t)$$

- The remaining cost from s_t to the goal using some policy π as

$$C^{\pi}(s_t)$$

Generic search

We define heuristic search in the context of shortest-path problems.
We now consider a general method for searching a node in the frontier.

input $G = \langle N, E \rangle$: Graph.
input $f : N \to \mathbb{R}$: evaluation function.
input x : Start node
function `Heuristic Search`(G, x, f)
$S' = \emptyset$: Nodes searched.
$F = \{x\}$. Initialise the frontier
$c_x = 0$. Initialise the cost of node x
while $F \neq \emptyset$ **do**
  $n = \arg\min_{i \in F} f(i)$. Select "best" node.
  $F = F \setminus \{n\}$. Remove n from the frontier.
  if $n \notin S'$ **then**
    $B = \mathtt{Ch}(n) \setminus S'$. Get the set of unsearched children of n.
    $\forall b \in B$, $b_i = c_n + c(n, b)$. Calculate the total cost to each child b.
    $S' = S' \cup \{n\}$. Add n to the list of searched nodes.
    $F = F \cup B$. Add n's children to the frontier.
  end if
end while

A^* search

We now consider a general method for searching a node in the frontier.

input $G = \langle N, E \rangle$: Graph.
input $h : N \to \mathbb{R}$: heuristic function.
input x : Start node
function $\texttt{A-Star}(G, x, h)$
$S' = \emptyset$ # Nodes searched.
$F = \{x\}$. # Initialise the frontier
$c_x = 0$. # Initialise the cost of node x
while $F \neq \emptyset$ **do**
  $n = \arg\min_{i \in F} c_i + h(i)$. # Select minimum cost + heuristic.
  $F = F \setminus \{n\}$. # Remove n from the frontier.
  if $n \notin S'$ **then**
    $B = \texttt{Ch}(n) \setminus S'$. # Get the set of unsearched children of n.
    $\forall b \in B$, $b_i = c_n + c(n, b)$. # Calculate the total cost to each child b.
    $S' = S' \cup \{n\}$. # Add n to the list of searched nodes.
    $F = F \cup B$. # Add n's children to the frontier.
  end if
end while

- You can see that $h = 0$ corresponds to minimum-cost search.

Admissible heuristics

- If h is arbitrary, then the search can fail.
- We need h to be admissible. In particular,

$$C^*(n) \geq h(n).$$

Admissibility of A^*

Theorem

A^* returns an optimal solution if

- The graph has a bounded branching factor.

Admissibility of A^*

Theorem

A^* returns an optimal solution if

- The graph has a bounded branching factor.
- All costs are greater than $\epsilon > 0$

Admissibility of A^*

Theorem

A^* returns an optimal solution if

- The graph has a bounded branching factor.
- All costs are greater than $\epsilon > 0$
- The heuristic is admissible, i.e. $0 \leq h(n) \leq C^*(n)$ for all $n \in N$.

Admissibility of A^*

Theorem

A^* returns an optimal solution if

- The graph has a bounded branching factor.
- All costs are greater than $\epsilon > 0$
- The heuristic is admissible, i.e. $0 \leq h(n) \leq C^*(n)$ for all $n \in N$.

Admissibility of A^*

Theorem

A^* returns an optimal solution if

- The graph has a bounded branching factor.
- All costs are greater than $\epsilon > 0$
- The heuristic is admissible, i.e. $0 \leq h(n) \leq C^*(n)$ for all $n \in N$.

Proof

- Existence. There is a finite number of paths that will be explored, as (a) the longest possible path to a goal is $C^*(0)/\epsilon$ and (b) the branching factor is bounded.

Admissibility of A^*

Theorem

A^* returns an optimal solution if

- The graph has a bounded branching factor.
- All costs are greater than $\epsilon > 0$
- The heuristic is admissible, i.e. $0 \leq h(n) \leq C^*(n)$ for all $n \in N$.

Proof

- Existence. There is a finite number of paths that will be explored, as (a) the longest possible path to a goal is $C^*(0)/\epsilon$ and (b) the branching factor is bounded.
- Optimality. The proof is by contradiction. Let as assume that A^* finds some $\pi \neq \pi^*$ so that $C(\pi) > C(\pi^*) \Rightarrow$ there is a node n on the path where a^* is optimal policy, but we instead follow π to $s_1, s_2, \ldots$. However, since $C(\pi) > C(\pi^*)$ there must be some t such that $C(n, s_1, \ldots, s_t) > C^{\pi^*}(n)$. But then, to expand π requires that $C(n, s_1, \ldots, s_t) + h(s') < h(s) \leq C^{\pi^*}(n)$.

Calculating Upper and Lower Bounds

Starting from a set of leaf nodes S_0

Calculating Upper and Lower Bounds

Starting from a set of leaf nodes S_0

Upper bound $U(s) \geq C^*(s)$ for $s \in S_0$

Setting $U(0) \geq C^*(0)$ and recursing:

Calculating Upper and Lower Bounds

Starting from a set of leaf nodes S_0

Upper bound $U(s) \geq C^*(s)$ for $s \in S_0$

Setting $U(0) \geq C^*(0)$ and recursing:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)]$$

Calculating Upper and Lower Bounds

Starting from a set of leaf nodes S_0

Upper bound $U(s) \geq C^*(s)$ for $s \in S_0$

Setting $U(0) \geq C^*(0)$ and recursing:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)]$$

By induction, we can prove that this is an upper bound on C^*:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)] \geq \min_{a \in A_s} c(s, a) + C^*[\tau(s, a)] = C^*(s).$$

Calculating Upper and Lower Bounds

Starting from a set of leaf nodes S_0

Upper bound $U(s) \geq C^*(s)$ for $s \in S_0$

Setting $U(0) \geq C^*(0)$ and recursing:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)]$$

By induction, we can prove that this is an upper bound on C^*:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)] \geq \min_{a \in A_s} c(s, a) + C^*[\tau(s, a)] = C^*(s).$$

Lower bound $L(s) \leq C^*(s)$ for $s \in S_0$

$$L(s) = \min_{a \in A_s} c(s, a) + L[\tau(s, a)]$$

Calculating Upper and Lower Bounds

Starting from a set of leaf nodes S_0

Upper bound $U(s) \geq C^*(s)$ for $s \in S_0$

Setting $U(0) \geq C^*(0)$ and recursing:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)]$$

By induction, we can prove that this is an upper bound on C^*:

$$U(s) = \min_{a \in A_s} c(s, a) + U[\tau(s, a)] \geq \min_{a \in A_s} c(s, a) + C^*[\tau(s, a)] = C^*(s).$$

Lower bound $L(s) \leq C^*(s)$ for $s \in S_0$

$$L(s) = \min_{a \in A_s} c(s, a) + L[\tau(s, a)]$$

Similarly, we can prove that it is a lower bound:

$$L(s) = \min_{a \in A_s} c(s, a) + L[\tau(s, a)] \leq \min_{a \in A_s} c(s, a) + C^*[\tau(s, a)] = C^*(s)$$

Branch and bound

The algorithm is rather simple to describe in words.

- [1] Set $s = 0$.
- [1.1] Select action a^* minimising $c(s, a) + L(\tau(s, a))$.
- [1.2] Discard subtrees (s, a) for which $c(s, a) + L(\tau(s, a)) \geq c(s, a^*) + L(\tau(s, a^*))$.
- [1.3] Proceed to $s = \tau(s, a)$ and go to 1.1. unless we are at a leaf.
- [2] Expand the leaf node, and generate new leaf nodes with corresponding upper and lower bounds.
- [3] Calculate L, S for the corresponding subtree.
- [4] Go to 1.

General weight shortest path

- In this problem, actions can have positive or negative costs.

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

Bellman-Ford Algorithm

In state-action notation, the algorithm is simply

- $C_0(0) = 0$, $C_0(s) = \infty$ for all $s \neq 0$.

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

Bellman-Ford Algorithm

In state-action notation, the algorithm is simply

- $C_0(0) = 0$, $C_0(s) = \infty$ for all $s \neq 0$.
- For $k \in 1, \ldots, |S| - 1$:

$$\forall s \in S : C_k(s) = \min_a c(s, a) + C_{k-1}(\tau(s, a))$$

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

Bellman-Ford Algorithm

In state-action notation, the algorithm is simply

- $C_0(0) = 0$, $C_0(s) = \infty$ for all $s \neq 0$.
- For $k \in 1, \ldots, |S| - 1$:

$$\forall s \in S : C_k(s) = \min_a c(s, a) + C_{k-1}(\tau(s, a))$$

- If $C_{|S|-1}(s) \neq \min_a c(s, a) + C_{|S|-1}(\tau(s, a))$ for some s, error

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

Bellman-Ford Algorithm

In state-action notation, the algorithm is simply

- $C_0(0) = 0$, $C_0(s) = \infty$ for all $s \neq 0$.
- For $k \in 1, \ldots, |S| - 1$:

$$\forall s \in S : C_k(s) = \min_a c(s, a) + C_{k-1}(\tau(s, a))$$

- If $C_{|S|-1}(s) \neq \min_a c(s, a) + C_{|S|-1}(\tau(s, a))$ for some s, error

General weight shortest path

- In this problem, actions can have positive or negative costs.
- Negative edges generate problems if we have cycles
- However, the basic algorithmic idea is again Dynamic Programming

Bellman-Ford Algorithm

In state-action notation, the algorithm is simply

- $C_0(0) = 0$, $C_0(s) = \infty$ for all $s \neq 0$.
- For $k \in 1, \ldots, |S| - 1$:

$$\forall s \in S : C_k(s) = \min_a c(s, a) + C_{k-1}(\tau(s, a))$$

- If $C_{|S|-1}(s) \neq \min_a c(s, a) + C_{|S|-1}(\tau(s, a))$ for some s, error

Difference with Dijkstra

- Bellman-Ford loops over all states in every iteration.
- Dijkstra only chooses a state in the frontier.
- It can handle negative costs and detect infinite cycles.
- Instead of tracking the frontier, it initialises C to infinity.

Bellman-Ford Algorithm

Here is a full pseudo-code for the Bellman-Ford algorithm

```
C(0) = 0. C(i) = ∞, for i ≠ 0.
for n ∈ 1, ..., |S| − 1 do
   # implement ∀j ∈ S: C(j) = min_i c(i,j) + C(i)
   for all edges (i,j) do
      if C(i) + c(i,j) < C(j) then
         C(j) = C(i) + c(i,j)
      end if
   end for
end for
# Cycle detection
for all edges (i,j) do
   if C(i) + c(i,j) < C(j) then
      error "Negative cycle"
   end if
end for
```